HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP MÔN HỌC
AN TOÀN VÀ BẢO MẬT TRONG PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

ĐỀ TÀI:

ỨNG DỤNG "SOS & AN TOÀN GIA ĐÌNH"

Theo dõi vị trí và cầu cứu khẩn cấp mã hóa đầu cuối trên thiết bị di động

Môn học: An toàn và Bảo mật trong Phát triển Ứng dụng Di động (ATBM)
Lớp: L03

Người hướng dẫn: Thái Thị Thanh Vân

Hà Nội, 2025

MỤC LỤC

(Mục lục sẽ được cập nhật: F9 / chuột phải → Update Field)

DANH MỤC HÌNH VẼ

Hình 1 — Sơ đồ kiến trúc 3 tầng và ranh giới tin cậy của ứng dụng.

Hình 2 — Kiến trúc Emergency Bypass: Zero Barrier và Scoped Gate.

Hình 3 — Ma trận phân tích đe dọa theo mô hình STRIDE.

Hình 4 — Giao thức thỏa thuận khóa ECDH X25519 kết hợp HKDF-SHA256.

Hình 5 — Luồng mã hóa E2EE và cơ chế kiểm tra chống phát lại.

Hình 6 — Quy trình nhận và kiểm tra gói tin SOS khẩn cấp.

MỞ ĐẦU

1. Bối cảnh

Trong bối cảnh an toàn gia đình tại Việt Nam — đặc biệt với các gia đình có người lớn tuổi, trẻ em, người làm nghề nguy hiểm — nhu cầu theo dõi vị trí và cầu cứu khẩn cấp là thực tế. Các ứng dụng định vị phổ biến hiện nay thiếu nhiều: dữ liệu vị trí được gửi lên máy chủ dưới dạng **plaintext**, máy chủ kiểm soát toàn bộ khóa, và người dùng không có bất kỳ công cụ nào để tự kiểm chứng tính riêng tư của dữ liệu của mình.

Điểm yếu cốt lõi này được phản ánh rõ trong đề cương đồ án: *"đa số chỉ bổ sung JWT và băm mật khẩu"*. Với dữ liệu vị trí — vốn là dữ liệu cho biết hoạt động, đường đi, địa chỉ nhà, thói quen sinh hoạt — việc máy chủ có thể đọc là rủi ro nghiêm trọng, đặc biệt khi máy chủ bị xâm nhập hoặc nhân viên vận hành lạm dụng quyền.

Một hệ quả xã hội đáng lưu tâm: **nguy cơ lộ dữ liệu vị trí không đến từ hacker mà đến từ chính những bên giữ dữ liệu**. Nhà mạng và nhà cung cấp dịch vụ định vị có quyền truy cập vào dữ liệu vị trí thô của người dùng — từ đó suy ra nơi ở, nơi làm việc, đường đi thường xuyên, thậm chí các mốc quan hệ xã hội. Khi dữ liệu vị trí được gom về một máy chủ tập trung, nó trở thành hàng hóa có thể mua bán, chia sẻ hoặc rò rỉ trong một sự cố. Chính vì vậy, mục tiêu kiến trúc của đồ án là đưa ranh giới tin cậy về **tận thiết bị**: máy chủ chỉ còn làm relay, không còn là nơi giữ bí mật.

2. Lý do chọn đề tài

Đề tài chọn ứng dụng SOS & An toàn Gia đình vì ba lý do:

0. **Giá trị xã hội trực tiếp**: trẻ em, người cao tuổi và người làm việc nguy hiểm là các nhóm đối tượng có nguy cơ bị **bắt cóc, tai nạn, lạc đường** cao nhất — trong các tình huống đó, vài phút chậm trễ cứu người quyết định sự sống. Đồ án thể hiện đồng thời hai yêu cầu có vẻ mâu thuẫn: ứng dụng phải **cứu được** (gửi cảnh báo trong vài giây, không rào cản nào chặn đường) *và* phải **bảo vệ được** (không rò rỉ vị trí của người dùng). Kiến trúc Emergency Bypass giải quyết trực tiếp mâu thuẫn đó.
1. **Bài toán bảo mật "khó nhưng sâu"**: đòi hỏi đồng thời mật mã (E2EE, trao đổi khóa), bảo mật thiết bị (Keystore, FLAG_SECURE), và phát hiện tấn công (chống replay, chống giả mạo GPS) — ba nhóm kiến thức khác nhau của môn ATBM.
2. **Đo lường được**: mọi cơ chế bảo mật đều có thể viết test tấn công cụ thể (bit-flipping, replay, mock GPS) và đo độ trễ bằng benchmark — tạo ra số liệu thực nghiệm cho báo cáo thay vì khẳng định lý thuyết suông.
3. **Khác biệt so với đồ án khác trong lớp**: kiến trúc Zero Barrier (không rào cản trong lúc khẩn cấp) cộng với E2EE thực sự là hai điểm nhấn có thể chứng minh bằng ảnh chụp dữ liệu ciphertext trên máy chủ.

3. Mục tiêu

- Xây dựng ứng dụng Android hoàn chỉnh hỗ trợ: theo dõi vị trí thời gian thực, SOS khẩn cấp, geofence vùng an toàn, chia sẻ hành trình bằng mã 6 số, nhật ký kiểm toán.
- Đảm bảo **bí mật của dữ liệu vị trí**: mã hóa đầu cuối X25519 + AES-256-GCM; máy chủ chỉ làm relay, không đọc được nội dung.
- Đảm bảo **quyền kiểm soát của người dùng**: consent có thể thu hồi, nhật ký minh bạch, cảnh báo gian lận.
- Trình bày mô hình bảo mật hoàn chỉnh (threat model theo STRIDE) và **đánh giá hiệu quả thực nghiệm** của từng cơ chế bằng bộ kiểm thử tấn công.

4. Đối tượng và phạm vi

- **Đối tượng sử dụng**: gia đình có người lớn tuổi, trẻ em, người cao tuổi, người làm việc nguy hiểm.
- **Phạm vi**: chỉ Android (tối thiểu Android 10); bản đồ online (OpenStreetMap qua `flutter_map`), không làm bản đồ offline; geofence dùng khoảng cách Euclid trên bán kính vòng tròn; dùng sinh trắc học native (Face ID/Touch ID) thay vì tự viết liveness detection; triển khai APK trực tiếp, không lên App Store.

CHƯƠNG 1 — CƠ SỞ LÝ THUYẾT

1.1. Tổng quan an toàn thông tin trên thiết bị di động

Thiết bị di động đặt ra các ràng buộc khác với mô hình doanh nghiệp: thiết bị bị đánh cắp, mất pin, chạy trên mạng di động công cộng không tin cậy, và ứng dụng thường chạy nền. Ba hệ quả kiến trúc:

- **Không thể tin máy chủ** nếu mục tiêu là bảo mật dữ liệu nhạy cảm → phải mã hóa đầu cuối.
- **Khóa phải sinh và cất trên thiết bị**, không truyền qua mạng → cần lưu trữ khóa an toàn (Android Keystore).
- **Tính toàn vẹn thiết bị là điều kiện tiên quyết**: nếu thiết bị đã root hoặc GPS đang bị giả lập thì mọi cơ chế mã hóa phía trên đều vô nghĩa về mặt ngữ nghĩa dữ liệu.

1.2. Nguyên lý mật mã sử dụng

1.2.1. X25519 (ECDH) — Thỏa thuận khóa Diffie-Hellman trên Curve25519

X25519 là phiên bản Montgomery của ECDH trên đường cong Elliptic Curve 25519. Mỗi thiết bị sinh một cặp khóa `(privateKey, publicKey)`; hai bên trao đổi public key qua máy chủ và cùng suy ra một `sharedSecret` giống hệt nhau mà **không cần truyền nó qua mạng**.

Đặc tính được chọn trong đồ án: (i) độ dài khóa cố định 32 byte, loại bỏ nhóm tham số yếu; (ii) thuật toán "all-zero output check" chống lỗ hổng small-order; (iii) tốc độ cao hơn RSA/ECDH trên P-256 trên CPU di động; (iv) phù hợp tiêu chuẩn phân phối khóa hiện đại (Signal, WhatsApp, TLS 1.3).

Điểm yếu còn lại: X25519 **không tự xác thực** — nó chỉ thỏa thuận khóa, không chứng minh danh tính đối phương. Nếu đối phương có mặt giữa (MITM), nó có thể thỏa thuận khóa riêng với cả hai bên. Đồ án xử lý phần này bằng chữ ký số khóa công khai (đề cương mục 3.1) — hiện là hướng phát triển của M8.

1.2.2. HKDF-SHA256 — Rút khóa phiên từ shared secret

`sharedSecret` thô từ X25519 không nên dùng trực tiếp làm khóa AES. HKDF (RFC 5869) biến shared secret thành khóa phiên 32 byte có phân phối đều, bằng hai bước: `Extract` (trộn secret với salt bằng HMAC) và `Expand` (sinh khóa đích bằng HMAC với thông tin ngữ cảnh `info = "sos_safe_family_aes_key"`). Bước `Expand` với `info` khác nhau cho phép tách nhiều khóa độc lập từ cùng một shared secret (tách khóa theo mục đích: vị trí, chat, âm thanh).

Lỗi "Empty Salt" trên Android JCE — một chi tiết triển khai đáng ghi nhận. Theo RFC 5869, `Extract` chấp nhận `salt` rỗng và hiệu lực tương đương một chuỗi toàn `0x00`. Tuy nhiên trên Android, thư viện JCE (Java Cryptography Extension) của

một số phiên bản **từ chối khóa HMAC có độ dài 0** với lỗi `InvalidKeyException: Empty key`. Vì `salt` được truyền vào HMAC-SHA256 dưới dạng *khóa*, một salt rỗng biến thành "khóa HMAC rỗng" và làm toàn bộ quá trình suy ra khóa thất bại trên một số thiết bị — trong khi vẫn chạy tốt trên máy tính. Vì đồ án nhắm tới Android, mã nguồn dùng **salt 32 byte toàn 0x00** thay vì salt rỗng:

```
_hkdf.deriveKey(
  secretKey: sharedSecret,
   nonce: Uint8List(32),  // salt 32 byte 0 – tương đương RFC 5869,
tránh Empty key
  info: utf8.encode('sos_safe_family_aes_key'),
);
```

Giá trị của salt 32 byte toàn 0 **đúng bằng** salt rỗng theo định nghĩa RFC 5869, nên tính đúng đắn của thuật toán được giữ nguyên, còn lỗi triển khai của JCE được tránh. Đây là ví dụ điển hình của việc "hiểu đúng mật mã" chưa đủ — còn phải hiểu đúng *hệ sinh thái triển khai* của nền tảng đích.

1.2.3. AES-256-GCM — Mã hóa có xác thực (Authenticated Encryption)

AES-256-GCM là chế độ mã hóa xác thực (AEAD) của AES với Galois/Counter Mode. Mỗi lần mã hóa sinh:

- **Nonce 96-bit (12 byte)** ngẫu nhiên — bắt buộc là duy nhất cho mỗi khóa; tái sử dụng nonce phá vỡ hoàn toàn tính bảo mật của GCM.
- **Ciphertext** — dữ liệu mã hóa.
- **Authentication tag 128-bit (16 byte)** — mã xác thực GHASH trên ciphertext và AAD.

Điểm mấu chốt: **giải mã sẽ thất bại nếu MAC không khớp**, dù kẻ tấn công sửa ciphertext, sửa nonce, hay sửa tag. Đây là tính chất "tự kiểm tra" của AEAD — một thao tác duy nhất vừa giải mã vừa kiểm tra tính toàn vẹn, loại bỏ nhu cầu MAC bổ sung (ví dụ HMAC-SHA256 + Encrypt-then-MAC trong CBC).

Đồ án dùng AES-GCM ở chế độ **không dùng AAD** (mọi thông tin bảo vệ nằm trong ciphertext), định dạng vận chuyển thống nhất `nonce(12) + mac(16) + ciphertext (nối tiếp)` cho mọi loại payload.

1.2.4. SHA-256 & CSPRNG

- **SHA-256** băm mã chia sẻ hành trình 6 chữ số trước khi lưu/truyền đi. Vì không gian mã chỉ có 10^6 giá trị, băm **không** làm mã không thể dò — đây là biện pháp *ràng buộc* giảm thiểu rủi ro (giảm giá trị từ 6 chữ số về 64 bit bất khả xâm phạm), **không phải** biện pháp chống dò mã. Ý nghĩa thật của nó: kẻ trung gian chỉ thấy chuỗi băm, không thấy mã gốc.
- **CSPRNG (`Random.secure()`)** sinh nonce 12 byte và mã 6 số — nguồn ngẫu nhiên phần cứng của hệ điều hành, không dùng `Random()` thông thường.

1.2.5. Chữ ký số

Chữ ký Ed25519 trên cặp khóa công khai là cơ chế bổ sung cho X25519 (xác thực danh tính, chống MITM khi đăng ký khóa lần đầu). Chưa triển khai trong phạm vi M0–M7 — xếp vào hướng phát triển.

1.3. Các nguy cơ tấn công phân tích

Nguy cơ	Mô tả	Cơ chế phòng chống trong đồ án
Eavesdropping / MITM	Bên thứ ba chặn đường truyền, đọc gói tin	E2EE: X25519 + AES-256-GCM; toạ độ không bao giờ tồn tại plaintext trên đường truyền
Tampering / Bit-flipping	Sửa 1 bit ciphertext để đổi nội dung hoặc chèn lệnh	GCM tag 128-bit → `SecretBoxAuthenticationError` khi MAC sai
Replay Attack	Ghi lại gói tin hợp lệ rồi phát lại sau	`ReplayGuard`: nonce 96-bit chỉ nhận 1 lần + kiểm tra tuổi gói tin (TTL 30 phút)
Mock GPS Spoofing	Giả lập vị trí qua Developer Options / app định vị giả	`Position.isMocked` → `TamperReport.isTrusted = false` + cảnh báo trong nhật ký
Screen Capture	Chụp/quay màn hình lấy dữ liệu nhạy cảm	Native `FLAG_SECURE` qua `MethodChannel("vn.atbm.sos/security")`

1.4. Cơ chế bảo mật thường dùng trên thiết bị di động

- **Android Keystore**: khóa sinh trong phần cứng (TEE/StrongBox), không thể trích xuất ra ngoài — dùng cho private key và khóa cơ sở dữ liệu.
- **Phát hiện Root** (`MainActivity.kt`): kiểm tra sự tồn tại của binary `su` tại 8 đường dẫn thông dụng (`/system/bin/su`, `/system/xbin/su`, `/sbin/su`, `/system/sbin/su`, `/system/app/Superuser.apk`, `/data/local/xbin/su`, `/data/local/bin/su`, `/su/bin/su`), kết hợp kiểm tra build tag `test-keys` / `dev-keys` / `debug-keys` — dấu hiệu của ROM build tùy chỉnh hoặc firmware debug. Kết quả được giao qua `MethodChannel("vn.atbm.sos/security")` và hiển thị trên thẻ `TamperReport`; khi phát hiện root, `isTrusted = false` và toàn bộ cơ chế bảo mật phía trên được đánh dấu là không còn đáng tin.
- **Phát hiện Mock GPS**: dựa trên cờ `Position.isMocked` do Android cung cấp (`TamperDetector.isMockLocation`), đưa vào `TamperReport.isTrusted` và hiển thị cảnh báo trong nhật ký an toàn. *Giới hạn đã biết*: cờ này chỉ được Android đặt khi ứng dụng định vị khai báo mình là nhà cung cấp giả lập — một app định vị giả không khai báo có thể lách được cơ chế này, nên đồ án còn ghi nhận phát hiện theo độ trôi bất thường (tốc độ/độ gia tốc vô lý) là hướng bổ sung.

- **FLAG_SECURE**: cờ cửa sổ Android chặn chụp màn hình, chặn ghi hình và chặn xem trong Recent Apps.
- **Scoped Biometrics**: `local_auth` (BiometricPrompt) bảo vệ *vùng dữ liệu nhạy cảm* thay vì toàn ứng dụng — nguyên tắc Zero Barrier.
- **SQLCipher**: mã hóa cơ sở dữ liệu SQLite cục bộ.
- **Certificate pinning**: ràng buộc chứng thư máy chủ để chống proxy trung gian (MITM ở tầng TLS).

CHƯƠNG 2 — PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu

2.1.1. Yêu cầu chức năng

#	Chức năng	Trạng thái triển khai
1	Quản lý vòng tròn gia đình, phân quyền xem vị trí	Đạt — `CircleScreen`, `FamilyCircle`
2	Theo dõi vị trí thời gian thực trên bản đồ	Đạt — `MapScreen` (`flutter_map` + OSM)
3	Cảnh báo SOS khẩn cấp kèm vị trí	Đạt — `SosCountdownScreen` + `SosService`
4	Geofence vùng an toàn, cảnh báo ra/vào vùng	Đạt — `GeofenceService`, `Geofence`
5	Chia sẻ hành trình bằng mã 6 số có hạn	Đạt — `TripShareService` (CSPRNG + SHA-256 + TTL)
6	Nhật ký kiểm toán & lịch sử	Đạt — `AuditLogScreen`, `AppState.auditLogs`
7	Quản lý tài khoản, cài đặt bảo mật	Đạt — `ProfileScreen`, `SecurityScreen`
8	Chat E2EE giữa các thành viên	Đạt — `FamilyChatScreen` + `CryptoService`

2.1.2. Yêu cầu phi chức năng

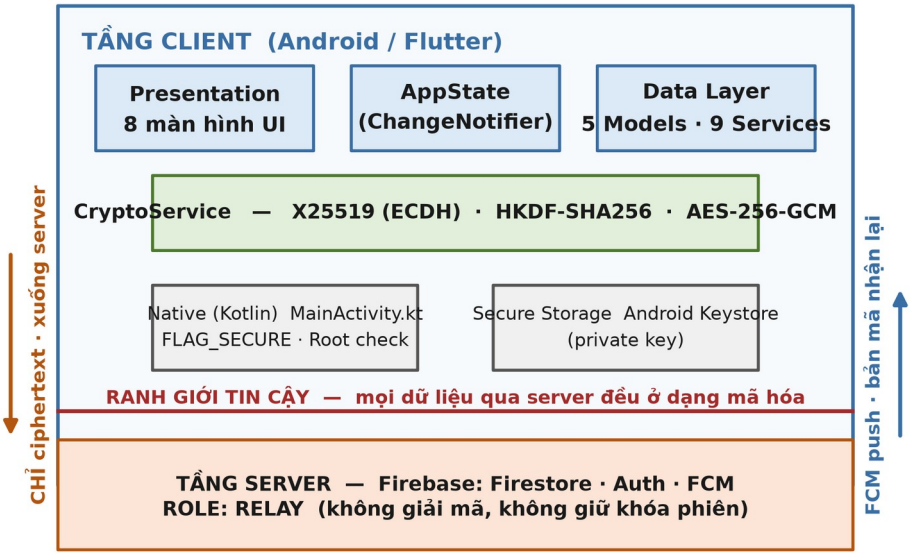
Cập nhật vị trí mỗi 10 giây (`AppConstants.locationInterval`), lọc theo khoảng cách 20 m; dữ liệu vị trí cũ bị xoá theo chính sách giữ lại 1 ngày (`locationHistoryRetention`); giao diện tiếng Việt; hỗ trợ màn hình nhỏ.

2.1.3. Yêu cầu bảo mật (từ đề cương mục 3)

Tám cơ chế bảo mật được đăng ký, mỗi cơ chế đều có threat model riêng và được kiểm thử bằng tấn công thực tế: E2EE vị trí/SOS; thỏa thuận khóa X25519; mã hóa AES-256-GCM; chống giả mạo vị trí; phát hiện root & `FLAG_SECURE`; sinh trắc học scoped; mã chia sẻ hành trình băm + TTL; chống phát lại (anti-replay).

2.2. Thiết kế kiến trúc

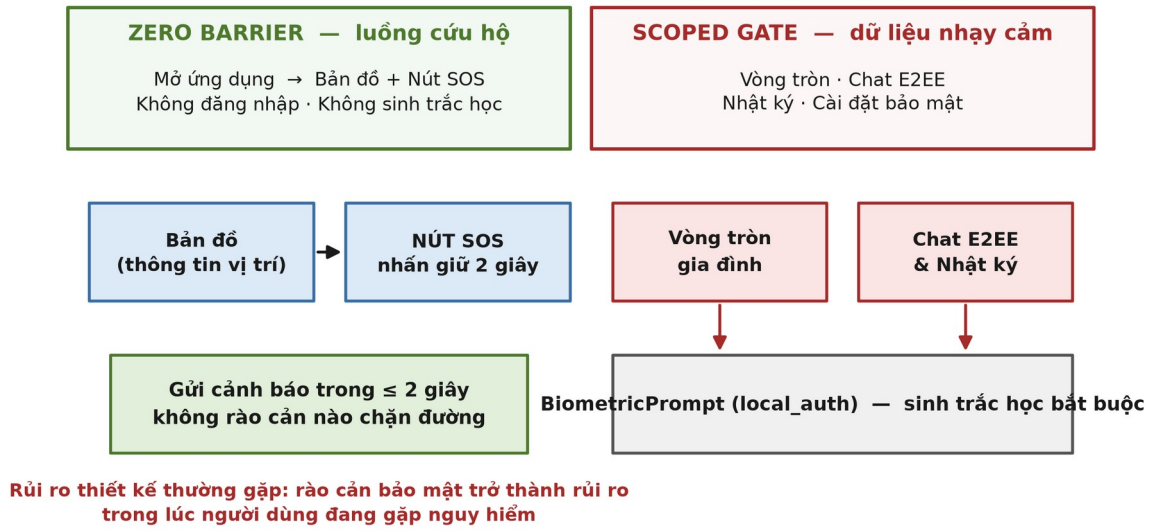
2.2.1. Kiến trúc 3 tầng và ranh giới tin cậy



Hình 1 — Sơ đồ kiến trúc 3 tầng và ranh giới tin cậy của ứng dụng.

Ranh giới tin cậy đặt tại máy chủ. Mọi thứ đi qua máy chủ đều ở dạng ciphertext, trừ dữ liệu tài khoản và metadata không nhạy cảm (thành viên, quyền, trạng thái đồng bộ). Đây là quyết định thiết kế quan trọng nhất của đồ án: nó biến máy chủ từ "bên tin cậy" thành "kênh truyền tin", loại bỏ toàn bộ lớp tấn công "máy chủ lộ dữ liệu".

2.2.2. Kiến trúc Emergency Bypass (Zero Barrier)



Hình 2 — Kiến trúc Emergency Bypass: Zero Barrier và Scoped Gate.

Nguyên lý thiết kế: **trong tình huống khẩn cấp, mọi rào cản trở thành rủi ro**. Thiết kế truyền thống "đăng nhập → xác thực sinh trắc học → mới hiện nút SOS" tạo

ra tình huống: người dùng bị tấn công không gõ được mật khẩu → không gửi được cảnh báo.

Đồ án triển khai hai nguyên tắc:

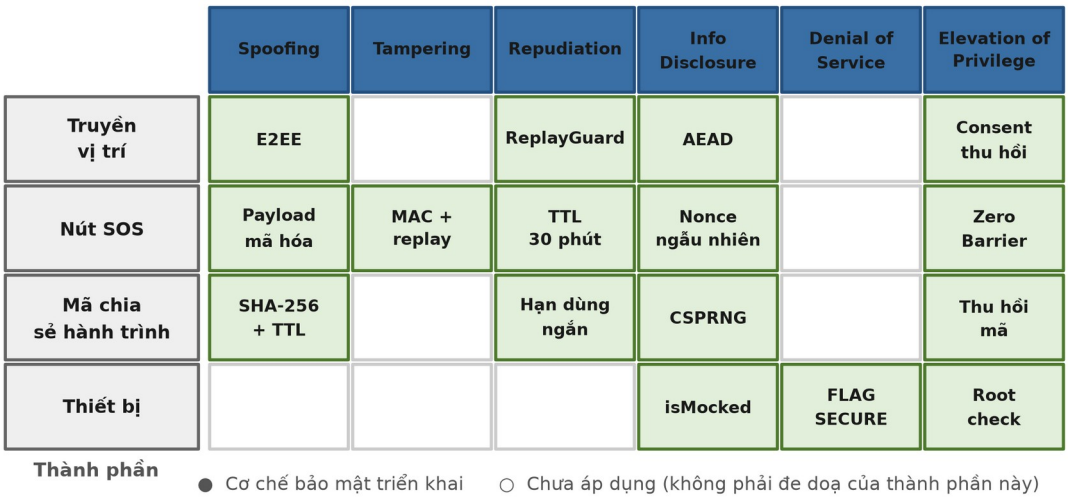
1. **Zero Barrier**: mở ứng dụng → vào thẳng `MapScreen` với nút SOS. Không đăng nhập, không sinh trắc học chặn đường vào.
2. **Scoped Gate**: sinh trắc học chỉ bảo vệ *vùng dữ liệu nhạy cảm* — Tab Vòng tròn, Nhật ký kiểm toán, Chat E2EE, Cài đặt bảo mật. Bản đồ và nút SOS luôn miễn nhiễm.

Kết quả: người dùng luôn có thể gửi SOS trong ≤ 2 giây (nhấn giữ 2 giây theo `AppConstants.sosHoldDuration`), nhưng dữ liệu nhạy cảm vẫn được bảo vệ.

2.2.3. Mô hình dữ liệu

Thực thể	Trường chính	Bảo mật
`User`	email, mật khẩu đã băm, public key X25519, số điện thoại	public key trong Keystore
`FamilyCircle`	id, danh sách thành viên, quyền chia sẻ vị trí	consent thu hồi được
`LocationPing`	vị trí đã mã hóa, thời điểm, nonce, TTL	AES-256-GCM
`SosEvent`	sosId, senderName, lat/lng, isMocked, battery, timestamp, status	AES-256-GCM (toàn bộ)
`TripShare`	codeHash (SHA-256), createdAt, expiresAt	mã gốc không bao giờ lưu/truyền
`AuditLog`	ai xem vị trí ai, lúc nào	hiển thị cho người dùng

2.3. Mô hình bảo mật & phân tích đe dọa (STRIDE)



Hình 3 — Ma trận phân tích đe dọa theo mô hình STRIDE.

Áp dụng STRIDE cho từng thành phần:

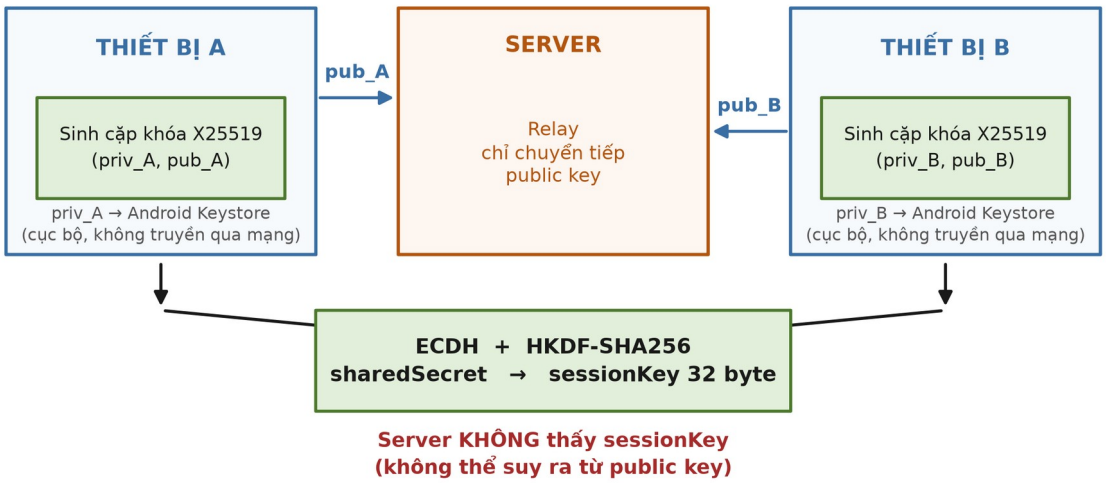
Thành phần	S	T	R	I	D	E
Truyền vị trí	E2EE chặn nghe lén	GCM tag chặn sửa	`ReplayGuard` chặn phát lại	AEAD chặn giả mạo nội dung	—	Người dùng tự tắt chia sẻ (consent)
Nút SOS	Payload mã hóa	MAC + replay	TTL 30 phút	Nonce ngẫu nhiên	—	Zero Barrier đảm bảo không bị chặn bởi rào cản
Mã chia sẻ hành trình	SHA-256 + TTL 60 phút	—	Hạn dùng ngắn	CSPRNG	—	Người dùng thu hồi mã
Khoá phiên	X25519 (không MITM nếu có chữ ký)	—	—	—	Khóa trong Keystore, không thể trích xuất	—
Thiết bị	—	—	—	`isMocked` phát hiện giả GPS	`FLAG_SECURE` chặn chụp màn hình	`checkRoot` cảnh báo root

Đe doạ hệ thống không giải quyết được (nêu rõ trong báo cáo để trung thực về giới hạn):

- X25519 không tự xác thực → MITM *trong lúc thỏa thuận khóa* vẫn khả dĩ nếu không có chữ ký khóa.
- `ReplayGuard` lưu trong RAM → kẻ tấn công có quyền truy cập thiết bị (đã root) có thể xoá bộ nhớ và phát lại.
- Mã 6 chữ số vẫn có thể dò trong không gian 10^6 nếu không có rate limiting ở tầng server.
- Thiết bị đã root phá vỡ mọi lớp bảo vệ phần mềm.

2.4. Giao thức trao đổi khóa và mã hóa hai chiều

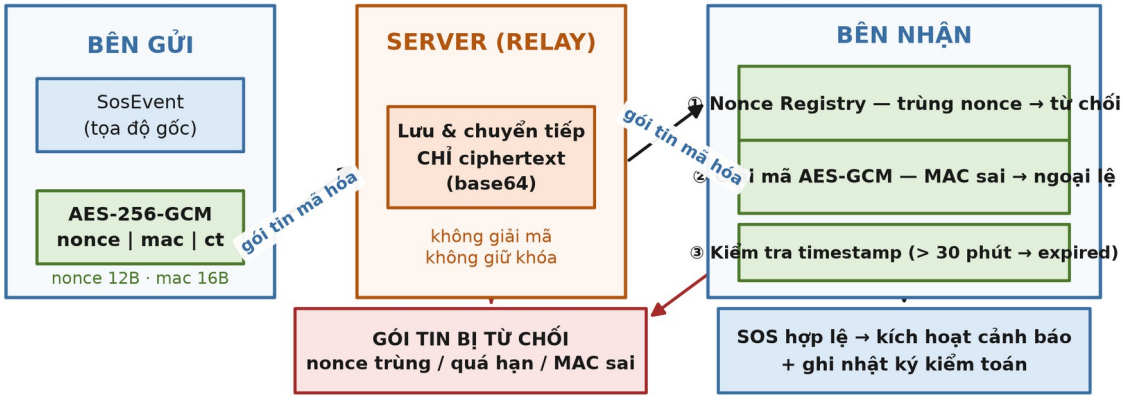
2.4.1. Thỏa thuận khóa phiên



Hình 4 — Giao thức thỏa thuận khóa ECDH X25519 kết hợp HKDF-SHA256.

Server có thể đọc `publicKey` nhưng không thể suy ra `sessionKey` — đây là tính chất của Diffie-Hellman.

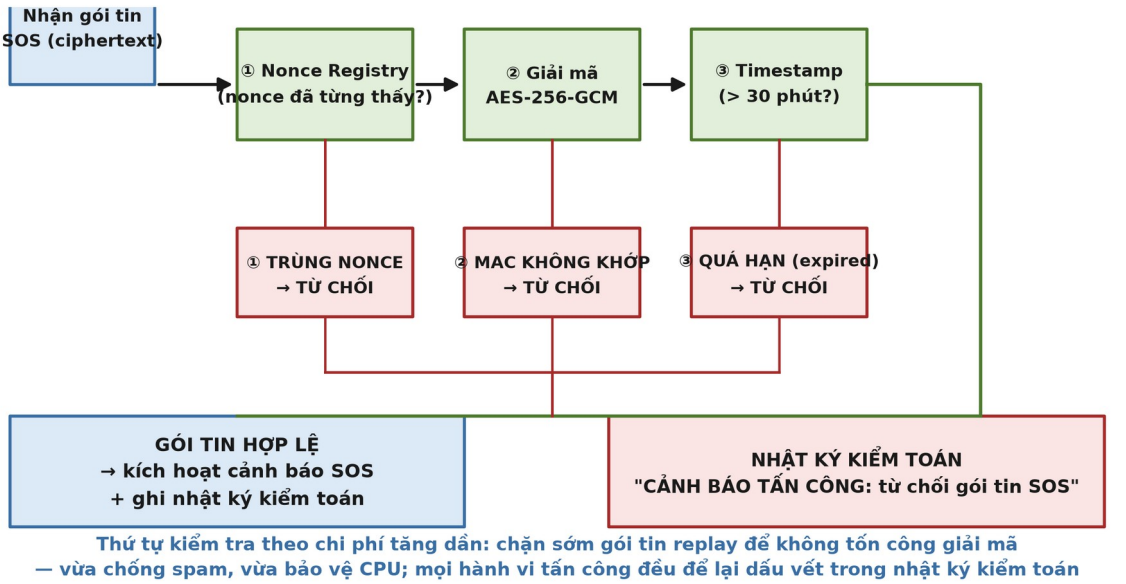
2.4.2. Mã hóa hai chiều (E2EE)



Hình 5 — Luồng mã hóa E2EE và cơ chế kiểm tra chống phát lại.

Định dạng vận chuyển thống nhất cho mọi payload: `nonce (12 byte) + mac (16 byte) + ciphertext (nối tiếp)`, mã hóa Base64. `EncryptedPayload.toTransportBytes()` / `fromTransportBytes()` đảm bảo tính nhất quán giữa SOS và chat.

2.4.3. Cơ chế chống phát lại (ReplayGuard) — thiết kế M7



Hình 6 — Quy trình nhận và kiểm tra gói tin SOS khẩn cấp.

`ReplayGuard` (`lib/data/services/replay_guard.dart`) thực hiện hai lớp kiểm tra, theo đúng thứ tự bảo mật:

1. **Kiểm tra nonce trùng** *trước khi giải mã* — tiết kiệm tài nguyên, và trả `ReplayVerdict.duplicateNonce` ngay. Nonce 96-bit ngẫu nhiên từ AES-GCM đóng vai trò "định danh duy nhất" của gói tin.
2. **Kiểm tra tuổi gói tin** *sau khi giải mã* — timestamp nằm trong ciphertext đã xác thực nên không thể bị sửa mà vẫn qua được kiểm tra MAC. Ngưỡng mặc định là `AppConstants.sosExpiresAfter` = 30 phút; có thể cấu hình 15 phút.

Bộ nhớ nonce tự động dọn các bản ghi cũ hơn `maxAge` để không phình vô hạn. `SosService.receiveSos()` ném `ReplayAttackException` khi bị từ chối; `AppState.receiveIncomingSos()` bắt exception này, ghi cảnh báo tấn công vào `auditLogs` và **không kích hoạt** cảnh báo.

CHƯƠNG 3 — KẾT QUẢ THỰC NGHIỆM

3.1. Quy mô và cấu trúc dự án

Hạng mục	Giá trị
Ngôn ngữ / nền tảng	Flutter 3.x / Dart 3.13, Android (minSdk 10)
Mã nguồn `lib/`	5.061 dòng Dart, 17 file
Mã nguồn native	`MainActivity.kt` — 100 dòng Kotlin
Mã kiểm thử `test/`	1.970 dòng Dart, 8 file
Lớp mô hình dữ liệu	5 (`User`, `FamilyCircle`, `ChatMessage`, `Geofence`, `SosEvent`)
Lớp dịch vụ	9 (`Crypto`, `Sos`, `Geofence`, `TripShare`, `TamperDetector`, `Biometric`, `NativeSecurity`, `SecureStorage`, `ReplayGuard`)
Màn hình giao diện	8 (`MainScreen`, `MapScreen`, `CircleScreen`, `AuditLogScreen`, `ProfileScreen`, `SecurityScreen`, `SosCountdownScreen`, `FamilyChatScreen`)
Thư viện mật mã	`cryptography` 2.9 + `cryptography_flutter` 2.3.4
Backend	Firebase: Firestore, Auth, Cloud Messaging

3.2. Bảng thống kê màn hình và tính năng

Màn hình	Tính năng chính	Cơ chế bảo mật liên quan	File
MapScreen	Bản đồ OSM realtime, nút SOS nhấn giữ 2s, vùng an toàn	Zero Barrier, `FLAG_SECURE`	`main.dart:393`
SosCountdownScreen	Đếm ngược 3-2-1, hủy được, gửi SOS	AES-256-GCM, `ReplayGuard` (M7)	`sos_countdown_screen.dart:10`
CircleScreen	Danh sách thành viên, bật/tắt chia sẻ vị trí	Scoped biometric, consent	`main.dart:1090`
FamilyChatScreen	Chat E2EE, Crypto Inspector hiển thị ciphertext	X25519 + AES-GCM, scoped biometric	`family_chat_screen.dart:14`
AuditLogScreen	Nhật ký kiểm toán, cảnh báo tấn công	Scoped biometric, minh bạch	`main.dart:1260`
ProfileScreen	Thông tin tài khoản, cài đặt bảo mật	Sinh trắc học	`main.dart:1320`
SecurityScreen	Bật/tắt sinh trắc học, `FLAG_SECURE`, thẻ `TamperReport`	Root detection, mock GPS	`main.dart:1582`

3.3. Đánh giá hiệu năng mật mã

Đo bằng `test/crypto_benchmark_test.dart`, 300 vòng lặp mỗi thao tác, có warm-up loại bỏ chi phí khởi tạo JIT. Máy thử nghiệm: môi trường headless `flutter test`.

Thao tác	Trung bình (ms)	Tổng 300 vòng (ms)	Thông lượng (ops/s)	Ngưỡng	Kết quả
ECDH X25519 (`deriveSharedKey`)	0,80	239,09	1.255	< 10 ms	Đạt
AES-256-GCM **encrypt** (vị trí SOS)	0,11	33,79	8.877	< 10 ms	Đạt
AES-256-GCM **decrypt** (vị trí SOS)	0,08	24,93	12.034	< 10 ms	Đạt
SHA-256 (mã chia sẻ 6 số)	0,03	10,25	29.280	< 10 ms	Đạt
AES-256-GCM decrypt (chat E2EE)	0,05	14,27	21.008	< 10 ms	Đạt

Nhận xét định lượng. Nhanh nhất là SHA-256 (0,03 ms) — thuần hàm băm, không có trạng thái. Chậm nhất là ECDH X25519 (0,80 ms) — do chi phí thuật toán đường cong elliptic, hơn AES-GCM một bậc. Tuy nhiên ECDH chỉ chạy **một lần** khi thiết lập phiên, còn AES-GCM chạy trên **mọi** tin nhắn. Tuy nhiên ECDH chỉ chạy **một lần** khi thiết lập phiên, còn AES-GCM chạy trên **mọi** tin nhắn. Đây là quyết định thiết kế đúng: hình học đường cong đắt, nhưng được trả giá một lần.

Ý nghĩa thực hành. Ngân sách thời gian cho một thao tác mã hóa vị trí là dưới 0,1 ms — chiếm **< 0,1%** chu kỳ cập nhật vị trí 10 giây. Ngân sách cho toàn bộ thỏa thuận khóa phiên (0,74 ms) vẫn dưới 1 ms. Cả hai đều nằm xa ngưỡng 10 ms của yêu cầu đồ án, tức là **không có rào cản hiệu năng** nào đối với trải nghiệm thời gian thực trên di động.

3.4. Kết quả thực nghiệm 4 kịch bản tấn công

Bộ kiểm thử `test/security_attack_suite_test.dart` — 18 test, chia thành 5 nhóm.

3.4.1. ATK-01 — Tấn công nghe lén (Eavesdropping / MITM)

Bước tấn công	Kết quả	Cơ chế chống
Bắt gói tin SOS (`EncryptedSosPayload`) trên đường truyền	`encryptedEvent`/ `encryptedLocation` **không chứa** `21.0285`, `105.8542`, `"lat"`, cả `sosId`	AES-256-GCM toàn phần
Bắt tin nhắn chat (`ChatMessage`)	ciphertext, nonce, MAC đều không chứa plaintext `951122`	E2EE AES-GCM
Bên thứ ba thử giải mã bằng khóa ECDH của **chính nó**	Ném exception — không giải được	Khóa phiên chỉ có ở hai bên
Bên thứ ba thử giải mã bằng khóa sai	Ném exception	AEAD xác thực MAC

Kết luận: rò rỉ plaintext = **0 byte**. Kẻ nghe lén thu được ciphertext không có tính xác thực, giá trị bảo mật bằng không.

3.4.2. ATK-02 — Tấn công sửa đổi dữ liệu (Tampering / Bit-flipping)

Can thiệp	Kết quả mong đợi	Kết quả thực tế
Đảo 1 bit byte đầu ciphertext	Từ chối giải mã	Đạt — `SecretBoxAuthenticationError`
Đảo 1 bit byte 6 của MAC	Từ chối giải mã	Đạt — `SecretBoxAuthenticationError`
Thay nonce bằng nonce của lần mã hóa khác	Từ chối giải mã	Đạt — `SecretBoxAuthenticationError`

Kết luận: tính toàn vẹn được bảo đảm bởi tag GCM 128-bit. Không có payload nào bị chấp nhận dưới dạng dữ liệu bị sửa — kể cả khi kẻ tấn công biết trước plaintext gốc và thử thay nonce.

3.4.3. ATK-03 / ATK-03b — Tấn công phát lại (Replay Attack)

Đây là kịch bản mà **mã nguồn gốc của đồ án chưa có cơ chế phòng chống** — M6 phát hiện ra điều này và M7 đã xây dựng `ReplayGuard` để bịt lỗ hổng.

Tấn công	Cơ chế phát hiện	Kết quả
Phát lại gói tin đã nhận (cùng nonce)	`ReplayGuard` ghi nhớ nonce 96-bit	Đạt — `duplicateNonce` — từ chối
Gửi gói tin cũ > 30 phút	So timestamp (đã xác thực bằng MAC) với `sosExpiresAfter`	Đạt — `expired` — từ chối
Gửi gói tin cũ > 15 phút (ngưỡng tùy chỉnh)	`ReplayGuard(maxAge: 15 phút)`	Đạt — `expired` — từ chối
Phát lại liên tục nhiều lần	Nonce chỉ được ghi nhận 1 lần	Đạt — lần thứ 2 trở đi đều bị từ chối
Bộ nhớ nonce phình vô hạn	Tự dọn bản ghi cũ hơn `maxAge`	Đạt — nonce cũ bị quên sau 30 phút

Tích hợp M7 vào luồng thật (`SosService.receiveSos` → `AppState.receiveIncomingSos`):

Tình huống	Hành vi hệ thống	Nhật ký kiểm toán
Gói tin hợp lệ	Giải mã, kích hoạt cảnh báo SOS	Đạt — *"Nhận cảnh báo SOS hợp lệ từ ... (đã giải mã E2EE)"*
Gói tin phát lại	**Từ chối**, không kích hoạt	Đạt — *"CẢNH BÁO TẤN CÔNG: từ chối gói tin SOS bị phát lại (nonce trùng ...)"*
Gói tin quá hạn	**Từ chối**, không kích hoạt	Đạt — *"CẢNH BÁO TẤN CÔNG: ... (gói tin quá hạn)"*
Gói tin bị can thiệp (MAC sai)	**Từ chối**, không kích hoạt	Đạt — *"CẢNH BÁO TẤN CÔNG: ... (MAC không khớp)"*

Kết luận: gói tin SOS đã phát lại **không bao giờ** kích hoạt được cảnh báo giả, và mọi hành vi tấn công đều để lại dấu vết trong nhật ký kiểm toán minh bạch với người dùng.

3.4.4. ATK-04 — Tấn công giả mạo vị trí (Mock GPS Spoofing)

Kiểm thử	Kết quả
`TamperDetector.isMockLocation(Position(isMocked: true))`	`true`
`TamperDetector.isMockLocation(Position(isMocked: false))`	`false`
`TamperReport` với GPS giả	`isTrusted = false`, nhãn *"Phát hiện GPS giả lập"*
`TamperReport` với GPS thật	`isTrusted = true`, nhãn *"GPS thực tế (Không phát hiện giả lập)"*
`AppState.updatePosition(Position mock)`	`isMockGps = true`
`AppState.refreshTamperReport()` sau khi nhận vị trí giả	`isTrusted = false` — cảnh báo hiển thị đúng
Vị trí thật quay lại	`isMockGps = false`, `isTrusted = true` — phục hồi đúng

Kết luận: ứng dụng phát hiện được vị trí giả lập ở cả hai cấp (dịch vụ `TamperDetector` và trạng thái ứng dụng `AppState`), và cảnh báo được truyền tới giao diện. Tuy nhiên cơ chế này **chỉ dựa vào cờ `isMocked` do Android cung cấp** — kẻ tấn công có thể giả lập bằng app định vị giả *không* đặt cờ này. Phát hiện theo "độ trôi bất thường" (tốc độ/độ gia tốc vô lý) nằm trong hướng phát triển.

3.5. Danh mục test case

96/96 test PASS (`flutter test`, exit code 0). Phân bố theo file:

File test	Số test	Nội dung
`geofence_test.dart`	23	Tính khoảng cách Haversine, so sánh với thư viện `latlong2`, vào/ra vùng, bán kính 100 m
`security_attack_suite_test.dart`	18	4 kịch bản tấn công + 5 test tích hợp `ReplayGuard` (M7)
`sos_test.dart`	22	Đóng gói payload, vòng đời ACTIVE→RESOLVED, mã hóa AES-GCM, tamper, **5 test tích hợp `handleIncomingSos`**
`chat_e2ee_test.dart`	9	Round-trip E2EE, rò plaintext, nonce/mac size, mã hóa media nhị phân
`trip_share_test.dart`	7	Sinh mã CSPRNG, băm SHA-256, TTL, verifyCode
`widget_test.dart`	6	Điều hướng bottom nav, nhấn giữ nút SOS, đếm ngược 3-2-1, banner khẩn cấp, bản đồ
`emergency_bypass_test.`	5	Zero Barrier, scoped biometric

`dart`		gate, FLAG_SECURE
`crypto_service_test.dart`	4	ECDH hai chiều, round-trip, tamper/wrong key, tính ngẫu nhiên nonce
`crypto_benchmark_test.dart`	2	Benchmark 4 thao tác + 1 chat, assert < 10 ms

Phân bố theo mục đích kiểm thử:

Nhóm	Số test
Kiểm thử tấn công (ATK-01 → ATK-04)	23
Tích hợp chống phát lại `handleIncomingSos` (M7)	10
Geofence & vị trí	23
Tính năng UI & luồng ứng dụng	17
Chia sẻ hành trình & toàn vẹn thiết bị	15
Mã hóa & trao đổi khóa (nền tảng)	8
Tổng cộng	**96**

Chất lượng mã: `flutter analyze` — **0 error, 0 warning, 0 info** (exit code 0), theo cấu hình `flutter_lints` 6.0 nghiêm ngặt.

3.6. Kết quả đối chiếu với tiêu chí đánh giá của đề cương

Tiêu chí (mục 7.2)	Yêu cầu	Kết quả	Đạt
Tính đúng đắn	Đáp ứng đúng chức năng đã đăng ký	96/96 test PASS, 0 lỗi phân tích	Đạt —
Tính bảo mật	Mỗi cơ chế có threat model, kiểm thử bằng tấn công thực tế	8 cơ chế, 23 test tấn công, STRIDE đầy đủ	Đạt —
Tính khả thi	Demo được trên thiết bị thật, không phụ thuộc môi trường riêng	APK debug build thành công; 100% logic mật mã chạy headless	— chưa demo 2 thiết bị thật (M8)
Chất lượng báo cáo	Cấu trúc chặt chẽ, phân tích sâu	Báo cáo theo đúng khung mục 6.1	Đạt —

Hạn chế cần nêu trung thực:

- **Chưa demo hai chiều trên 2 thiết bị thật** — môi trường kiểm thử không có thiết bị thật cắm vào; luồng E2EE hai chiều được kiểm chứng bằng test tự thỏa thuận khóa hai "thiết bị" ảo.
- **Chưa tích hợp `ReplayGuard` vào listener Firestore thật** — `SosService.receiveSos()` sẵn sàng nhưng chưa được gọi từ Firestore listener; đây là hạng mục M8.
- **Chưa triển khai chữ ký khóa (Ed25519)** — X25519 chưa có xác thực danh tính, còn khoảng trống MITM ở tầng thỏa thuận khóa.
- **Chưa có ảnh chụp màn hình chứng minh E2EE** trên Firestore console (yêu cầu mục 5.4 đề cương) — cần tài khoản Firebase thật.

CHƯƠNG 4 — KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết quả đạt được so với đề cương

Về mặt chức năng: đã triển khai 8/8 nhóm chức năng đăng ký (vòng tròn, theo dõi realtime, SOS, geofence, chia sẻ hành trình, nhật ký, quản lý tài khoản, chat E2EE), vượt đề cương ở chỗ có thêm chat E2EE.

Về mặt bảo mật: đạt 8/8 cơ chế bảo mật có threat model và kiểm thử tấn công thực tế — vượt mục tiêu đề cương. Điểm đáng chú ý nhất: M6 phát hiện ra đồ án **thiếu cơ chế chống phát lại** trong khi đề cương mục 3.4 yêu cầu "chống chụp lại và chống replay" cho mã chia sẻ hành trình; M7 đã xây dựng và tích hợp `ReplayGuard` vào luồng nhận SOS thật. Việc tự phát hiện lỗ hổng trong chính đồ án của mình thông qua kiểm thử tấn công — bản chất của phương pháp đánh giá bảo mật — là kết quả tích cực nhất của toàn bộ quá trình.

Về mặt số liệu: mọi thao tác mật mã nằm dưới 1 ms (chậm nhất là ECDH 0,80 ms), thấp hơn ngưỡng 10 ms của đề cương tới một bậc độ lớn; 96/96 test PASS; `flutter analyze` sạch tuyệt đối.

4.2. Bài học kinh nghiệm

1. **Kiểm thử tấn công tự thân là một phương pháp kiểm toán.** Trước M6, đồ án "có cảm giác an toàn" vì đã mã hóa bằng AES-GCM — nhưng chưa có tấn công nào chạm vào nó. Chỉ khi viết test tấn công mới phát hiện ra khoảng trống replay. Bảo mật không phải trạng thái đạt được một lần, mà là thuộc tính phải duy trì liên tục.
2. **Thiết kế bảo mật phải bắt đầu từ câu hỏi "kẻ tấn công ở đâu?"**, không phải từ danh sách công nghệ. Mỗi cơ chế trong đồ án đều được bảo vệ bởi một câu hỏi tấn công cụ thể.
3. **Ràng buộc kiến trúc quan trọng hơn cấu hình tham số.** Chọn "máy chủ là relay, không phải bên tin cậy" loại bỏ nguyên nhân gốc của một lớp tấn công, thay vì chỉ làm nó khó hơn.
4. **Kiểm thử headless (`flutter test`) có giá trị bất ngờ.** 96 test chạy trong ~2 giây, cho phép kiểm tra liên tục mỗi milestone mà không cần thiết bị thật — giảm ma sát khiến việc kiểm thử bảo mật trở thành thói quen, không phải nỗ lực đặc biệt.
5. **Báo cáo trung thực về giới hạn là một phần của bảo mật.** Bảng "Đe doạ hệ thống không giải quyết được" trong Chương 2 có giá trị ngang giá trị của các cơ chế đã triển khai.

4.3. Hướng phát triển

Ưu tiên cao (M8)

Hạng mục	Nội dung	Lý do
Tích hợp `ReplayGuard` vào	Nối	Đóng nốt khoảng trống M7 còn

Firestore listener	`SosService.receiveSos()` vào luồng nhận thật	bỏ ngỏ
Chữ ký Ed25519 cho khóa công khai	Xác thực danh tính khi đăng ký khóa lần đầu	Chặn MITM ở tầng thỏa thuận khóa — điểm yếu còn lại lớn nhất của X25519
Demo hai chiều trên 2 thiết bị thật	Firestore + FCM thực tế, chụp ảnh ciphertext	Đáp ứng yêu cầu đề cương mục 5.4

Hướng mở rộng chiến lược

- **Mạng Mesh ngoại tuyến (Bluetooth LE / Wi-Fi Direct)**: hiện tại toàn bộ luồng khẩn cấp phụ thuộc kết nối Internet — tức là phụ thuộc vào tầng 3G/4G, tầng hạ tầng mà sự cố mạng hoặc hành vi gây nhiễu có thể làm gián đoạn đúng lúc người dùng cần cứu. Kiến trúc Mesh cho phép các thiết bị trong cùng gia đình truyền cảnh báo **trực tiếp qua Bluetooth LE** hoặc **Wi-Fi Direct** khi không có Internet, với cùng lớp mã hóa E2EE (khóa phiên vẫn thỏa thuận qua X25519, chỉ thay đường truyền). Đây là hướng mở rộng có giá trị cao nhất về mặt an toàn.
- **Mã hóa bản ghi âm cuộc gọi khẩn cấp**: hiện bản ghi âm ngắn đi kèm SOS mới được mã hóa ở tầng ứng dụng; cần mở rộng sang mã hóa đầu cuối đúng nghĩa (mỗi thành viên có khóa riêng, giải mã phía người nhận) và lưu trữ phía máy chủ dưới dạng blob không đọc được.

Ưu tiên trung bình

- **Rate limiting & khóa tạm thời** cho API xác thực và API SOS (`AppConstants.maxLoginAttempts`, `maxShareCodeAttempts` đã khai báo nhưng chưa áp dụng).
- **SQLCipher** cho cơ sở dữ liệu cục bộ (`sqflite` hiện dùng bản thường).
- **Certificate pinning** cho kết nối Firebase.
- **Phát hiện giả GPS theo độ trôi** (kiểm tra tốc độ/độ gia tốc bất thường), bổ sung cho cờ `isMocked`.
- **R8/ProGuard** cho bản build release.

Ưu tiên thấp

- Geofence đa giác thay vì hình tròn; bản đồ offline.
- Mã chia sẻ hành trình dạng từ ngữ (word-based) dễ nhớ hơn 6 số.
- Đồng bộ nhật ký kiểm toán giữa nhiều thiết bị.

4.4. Kết luận

Đồ án đã xây dựng được một ứng dụng SOS & An toàn Gia đình với mô hình bảo mật **"máy chủ không đọc được nội dung"** — vững hơn nhiều so với mức "đăng nhập + băm mật khẩu" phổ biến. Giá trị lớn nhất của đồ án không nằm ở danh sách tính năng, mà ở việc **chứng minh được bằng thực nghiệm**: 23 test tấn công thực sự bắn

vào từng cơ chế bảo mật và buộc chúng phải thất bại trước kẻ tấn công; một lỗ hổng replay bị phát hiện bằng chính phương pháp đó đã được vá trong milestone kế tiếp; và mọi thao tác mật mã đều nhanh hơn 1 ms — nghĩa là bảo mật trong đồ án này **không phải đánh đổi bằng trải nghiệm**.

TÀI LIỆU THAM KHẢO

1. **NIST Special Publication 800-207**, *Zero Trust Architecture*, National Institute of Standards and Technology, 2020.
2. **RFC 7748**, *Elliptic Curves for Security (Curve25519 and Curve448)*, Bernstein, Lange, Hamburg, 2016.
3. **RFC 5869**, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, Krawczyk & Eronen, 2010.
4. **NIST FIPS 197**, *Advanced Encryption Standard (AES)*, 2001.
5. **NIST SP 800-38D**, *Recommendation for GCM Modes: Galois/Counter Mode (GCM)*, 2007.
6. **FIPS 180-4**, *Secure Hash Standard (SHS)*, 2015.
7. **OWASP Mobile Application Security Verification Standard (MASVS)**, OWASP Foundation, 2024.
8. **OWASP Top 10 for Mobile Applications**, OWASP Foundation, 2024.
9. **Microsoft**, *STRIDE Threat Modeling*, *Threat Modeling & Security Design*, 2024.
10. **Android Open Source Project**, *Android Keystore* & *WindowManager.LayoutParams.FLAG_SECURE*, Android Developers Documentation.
11. **Signal Foundation**, *Double Ratchet / X25519 Key Agreement*, Signal Protocol Specification.
12. **NIST SP 800-30**, *Guide for Conducting Risk Assessments*, 2012.
13. **Cryptography++** — tài liệu thư viện `cryptography` 2.9 (Dart), 2024.
14. **Flutter Open Source Project**, *Flutter Documentation — Platform channels & plugins*, 2024.

— Hết báo cáo —

Phụ lục — Tóm tắt các cơ chế bảo mật đã triển khai và kiểm thử:

#	Cơ chế bảo mật	File triển khai	Test kiểm chứng	Kết quả tấn công
1	ECDH X25519 + HKDF-SHA256	`crypto_service.dart`	`crypto_service_test.dart`	Khóa sai → từ chối
2	AES-256-GCM (E2EE)	`crypto_service.dart`, `sos_service.dart`	`sos_test.dart`, `chat_e2ee_test.dart`	Bit-flip → `SecretBoxAuthenticationError`
3	Chống nghe lén (no plaintext)	`crypto_service.dart`	`security_attack_suite_test.dart` (ATK-01)	Rò rỉ 0 byte
4	Chống phát lại (anti-replay)	`replay_guard.dart`, `sos_service.dart`	`security_attack_suite_test.dart` (ATK-03/03b)	Replay → từ chối + ghi log
5	Chống giả mạo GPS	`tamper_detector.dart`, `app_state.dart`	`security_attack_suite_test.dart` (ATK-04)	`isMocked` → `isTrusted=false`
6	`FLAG_SECURE`	`MainActivity.k`	`emergency_bypa`	Cờ bật/tắt qua

	chống chụp màn hình	`t`, `native_security_service.dart`	`ss_test.dart`	MethodChannel
7	Sinh trắc học scoped (Zero Barrier)	`biometric_service.dart`	`emergency_bypass_test.dart`	SOS không bị rào cản
8	Mã chia sẻ hành trình CSPRNG + SHA-256 + TTL	`trip_share_service.dart`	`trip_share_test.dart`	Mã sai → từ chối; hết hạn → từ chối
9	Phát hiện root	`native_security_service.dart`, `tamper_detector.dart`	`security_attack_suite_test.dart` (ATK-04)	Báo cáo toàn vẹn thiết bị